

网络知识点总结

Tcp三次握手与四次挥手

什么是TCP?

TCP是面向连接的、可靠的、基于字节流的传输层通信协议

面向连接:一定是一对一才能连接，不能像UDP协议可以一个主机同时向多个主机发送消息，也就是一对多是无法做到的

可靠的:无论的网络链路中出现了怎样的链路变化，TCP都可以保证一个报文一定能够到达接收端

字节流：用户消息通过TCP协议传输时，消息可能会被操作系统【分组】成多个TCP报文，如果接收方的程序如果不知道【消息的边界】，是无法读出一个有效的用户信息的。并且TCP报文时【有序的】，当【前一个】TCP报文没有收到的时候，即时它先收到了后面的TCP报文，那么也不能扔给应用层去处理，同时对【重复】的TCP报文会自动丢弃。

TCP头部中有哪些信息？

序号（32bit）：传输方向上字节流的字节编号。初始时序号会被设置一个随机的初始值（ISN），之后每次发送数据时，序号值 = ISN + 数据在整个字节流中的偏移。假设A -> B且ISN = 1024，第一段数据512字节已经到B，则第二段数据发送时序号为1024 + 512.用于解决网络包乱序问题

确认号（32bit）：接收方对发送方TCP报文段的响应，其值是收到的序号值+1

首部长（4bit）：标识首部有多少个4字节 * 首部长，最大为15，即60字节

标志位（6bit）：

- URG：标志紧急指针是否有效
- ACK：标志确认好是否有效（确认报文段）。用于解决丢包问题

- PSH: 提示接收端立即从缓冲区读走数据
- RST: 表示要求对方重新建立连接（复位报文段）
- SYN: 表示请求建立一个连接（连接报文段）
- FIN: 表示关闭连接（断开报文段）

窗口（16bit）：接收窗口。用于告知对方（发送方）本方的缓冲还能接收多少字节数据。用于解决流控

校验和（16bit）：接收端用CRC检验整个报文段有无损坏

UDP和TCP有什么区别呢？ 分别的应用场景是？

udp不提供复杂的控制机制，利用IP提供面向【无连接】的通信服务。

udp协议真的非常简，头部只有8个字节（64位），udp的头部格式如下：

目标和源端口：主要是告诉UDP协议应该把报文发给哪个进程。

包长度：该字段保存了UDP首部的长度跟数据的长度之和

校验和：校验和是为了提供可靠的UDP首部和数据而设计，防止收到在网络传输中受损的UDP包。

TCP和UDP的区别：

1.连接

- TCP是面向连接的传输层协议，传输数据前要先建立连接。
- UDP不需要连接，即刻传输数据

2.服务对象

- TCP是一对一的两点服务，即一条连接只有两个端点。
- UDP支持一对一、一对多、多对多的交互通信

3.可靠性

- TCP是可靠交付数据的，数据可以无差错、不丢失、不重复、按序到达。
- UDP是不可靠的传输协议，不保证可靠交付数据，数据发送在之后，丢失了就丢失了。但我们可以基于UDP传输协议实现一个可靠的传输协议，比如QUIC协议。

4.拥塞控制、流量控制

- TCP有拥塞控制和流量控制机制，保证数据传输的安全性
- UDP则没有，即使网络非常拥堵了，也不会影响UDP的发送速率。

5.首部开销

- TCP首部长度的较长，会有一定的开销，首部在没有使用【选项】字段时是20个字节，如果使用了【选项】字段则是会变长的
- UDP首部只有8个字节，并且是固定不变的，开销较小

6.传输方式

- TCP是流式传输，没有边界，但保证顺序和可靠
- UDP是一个包一个包的发送，是有边界的，但可能会丢包和乱序

7.分片不同

- TCP的数据大小如果大于MSS大小，则会在传输层进行分片，目标主机收到后，也同样在传输层组装TCP数据包。如果中途丢失了一个分片，只需要传输丢失的这个分片
- UDP的数据大小如果大于MTU大小，则会在IP层进行分片，目标主机收到后，在IP层组装完数据，接着再传给传输层。

TCP和UDP的应用场景

由于TCP是面向连接，能保证数据的可靠性交付，因此经常用于：

- FTP文件传输
- HTTP/HTTPS

由于UDP面向无连接，它可以随时发送数据，再加上UDP本身的处理既简单又高效，经常用于：

- 包总量较少的通信，如DNS、SNMP等；
- 视频、音频等多媒体通信；
- 广播通信

TCP和UDP可以使用同一个端口吗？

答案是：可以的

在数据链路层中，通过 MAC 地址来寻找局域网中的主机。在网际层中，通过 IP 地址来寻找网络中互连的主机或路由器。在传输层中，需要通过端口进行寻址，来识别同一计算机中同时通信的不同应用程序。

所以，传输层的「端口号」的作用，是为了区分同一个主机上不同应用程序的数据包。

传输层有两个传输协议分别是 TCP 和 UDP，在内核中是两个完全独立的软件模块。

当主机收到数据包后，可以在 IP 包头的「协议号」字段知道该数据包是 TCP/UDP，所以可以根据这个信息确定送给哪个模块（TCP/UDP）处理，送给 TCP/UDP 模块的报文根据「端口号」确定送给哪个应用程序处理。

因此，TCP/UDP各自的端口号也相互独立，如TCP有一个80端口，UDP也可以有一个80号端口，二者并不冲突。

什么是UDP？

提供无连接的，尽最大努力的数据传输服务（不保证数据传输的可靠性）

封包和拆包是基于TCP还是UDP？

封包和拆包都是基于TCP的概念。因为TCP是无边界的流传输，所以需要对TCP进行封包和拆包，确保发送和接收的数据不粘连。

- 封包：封包就是在发送数据报的时候为每个TCP数据包加上一个包头，将数据报分为包头和包体两个部分。包头是一个固定长度的结构体，里面包含该数据包的总长度
- 拆包：接收方在接收到报文后提取包头中的长度信息进行截取

TCP对应的应用层协议

FTP：定义了文件传输协议，使用21端口

Telnet：它是一种用于远程登录的端口，23号端口

SMTP: 定义了简单邮件传送协议，服务器开放的是25号端口

POP3: 它是和SMTP对应，POP3用于接收邮件

UDP对应的应用层协议

DNS: 用于域名解析服务，用的是53号端口

SNMP: 简单网络管理协议，使用161号端口

TFTP: 简单文件传输协议，使用69号端口

TCP连接建立

TCP三次握手流程？

。。。

注意：三次握手中，前两次握手不能携带数据，但是第三次握手可以携带数据

一旦完成三次握手，双方都处于 ESTABLISHED 状态，此时连接就已建立完成，客户端和服务端就可以相互发送数据了。

为什么是三次握手？不是两次、四次？

TCP建立连接时，通过三次握手能防止历史连接的建立，能减少双方不必要的资源开销，能帮助双方同步初始化序列号。序列号能够保证数据包不重复、不丢弃和按序传输。

不使用【两次握手】和【四次握手】的原因：

【两次握手】：无法阻止历史连接的建立，会造成双方资源的浪费，也无法可靠的同步双方序列号

【四次握手】：三次握手理论上就已经是最少可靠连接建立，所以不需要使用更多的通信次数

ISN（Initial Sequence Number）是固定的吗？

当一端为建立连接而发送他的SYN时，它为连接选择一个初始序号。ISN随时间而变化，因此每个连接都将具有不同的ISN，ISN是一个有可以看作是一个32bit的计数器，但并不是简单的计数器，大概每4ms加1。

```
ISN = M + F (localhost, localport, remotehost, remoteport) (M为计数器)，ISN应该由这个公式确定，F为哈希算法，不是一个简单计数器
```

这样选择序号的目的在于防止在网络中被延迟的分组在以后又被传送，而导致某个连接的一方对它做错误的解释。

三次握手的其中一个重要功能是客户端和服务端交换ISN，以便让对方知道接下来接收数据的时候如何按序列号组装数据。如果ISN是固定的，攻击者很容易猜出后续的确认证，因此ISN是动态生成的。

为什么每次建立TCP连接时，初始化的序列号都要求不一样呢？

主要原因有两个方面：

- 为了防止历史报文被下一个相同四元组的连接接收（主要方面）；
- 为了安全性，防止黑客伪造的相同序列号的TCP报文被对方接收

主要为了防止的情况：

客户端和服务端建立一个TCP连接，在客户端发送数据包被网络阻塞了，触发超时重传，然后服务端断电重启了，之前与客户端建立的连接就消失了，所以在接收到客户端的数据包后，给客户端发一个RST重新连接的请求。

然后，客户端与服务端又重新建立了遇上一个连接相同初始化序列的连接。

在新连接完事的时候，上一个超时重传的包正好到达服务端，那么此时这个包很大可能是在服务端的接收窗口内的，所以它会接受这个历史数据包，导致数据错乱

所以，如果每次建立连接客户端和服务端的初始化序列号都「不一样」，就有大概率因为历史报文的序列号「不在」对方接收窗口，从而很大程度上避免了历史报文

但并不是完全避免，还需要借助时间戳来判断历史报文

什么是SYN攻击？如何避免SYN攻击？

TCP的连接需要三次握手，假设攻击者短时间伪造不同的IP地址的SYN报文，服务端每接收到一个SYN报文，就进入SYN_recv状态，但服务端发出去的ACK+SYN报文，无法得到未知IP主机的ACK应答，久而久之就会占满服务端的半连接队列，使得服务端不能为正常用户服务。

什么是TCP半连接和全连接队列？

在TCP进行三次握手的时候，Linux内核会维护两个队列，分别是：

- 半连接队列，也成为SYN队列
- 全连接队列，也称为accept队列

正常流程：

- 当服务端接收到客户端的SYN报文时，会创建一个半连接的对象，然后将其加入到内核的「SYN队列」；
- 接着发送SYN + ACK给客户端，等待客户端回应ACK报文；
- 服务端接收到ACK报文后，从「SYN队列」取出一个半连接对象，然后创建一个新的连接对象放入到「Accept队列」；
- 应用通过调用accept() socket接口，从「Accept队列」取出连接对象。

不管是半连接队列还是全连接队列，都有最大长度限制，超过限制时，默认情况都会丢弃报文。

SYN攻击方式最直接的表现就会把 TCP半连接队列打满，这样当TCP半连接队列满了，后续再在收到SYN报文就会丢弃，导致客户端无法和服务端建立连接。

避免SYN攻击方式，可以有以下四种方法：

- 调大netdev_max_backlog（当网卡接受速度大于内核处理速度时，会将没处理的包放在这个队列中）
- 增大TCP半连接队列
- 开启tcp_syncookies
- 减少SYN+ACK重传次数（默认情况是5次，可以调小至两次）

TCP连接断开

四次挥手流程

- 第一次挥手：主动关闭方（以客户端为例）发送FIN报文，进入FIN_WAIT_1状态，表示“我已无数据发送，但还能接收数据”
- 第二次挥手：被动关闭方（服务端）收到FIN后，发送ACK确认报文，进入CLOSE_WAIT状态；客户端收到ACK后，进入FIN_WAIT_2状态
- 第三次挥手：服务端处理完剩余数据后，发送FIN报文，进入LAST_ACK状态，表示“我也无数据发送，准备关闭”
-

网络层

断网了，还能ping通127.0.0.1吗？

哪怕拔了网线，ping127.0.0.1是能ping通的

什么是127.0.0.1

这是一个IPV4地址，总共四个字节

其中127开头的都属于回环地址，也是IPV4的特殊地址（人为规定）

而127.0.0.1是众多回环地址中的一个。那为什么不是127.0.0.2呢，源码里就是这么规定的，也没什么道理

```
#define INADDR_LOOPBACK 0x7f000001 /*127.0.0.1*/
```

IPV4的地址是32位的，2的32次方，大概是40亿，不够全球人用，所以就有了IPV6，IPV6是128位的，大概是10的28次方，所以IPV6的IP可以认为用不完

IPV4以8位一组，每组之间用.号隔开。IPV6就以16位为一组，每组之间用:号隔开。如果全是0，那么可以省略不写。

在IPV4下的回环地址是 127.0.0.1，在IPV6下，表达为 ::1。它的完整写法是 0:0:0:0:0:0:0:1（共 7 个冒号），根据 RFC 4291，IPv6 地址中一段或多段连续的全零可以用 :: 压缩一次，因此可以把前面 7 段连续的 0 压缩掉，简写为 ::1；需要注意的是，一个 IPv6 地址中 :: 只允许出现一次，否则会产生歧义。

在**IPV4**下用的是 `ping 127.0.0.1`，而在**IPV6**下用的是 `ping6 ::1` 命令

什么是ping

ping是应用层命令，ping作为一个小软件，它的功能比较简单，就是尝试发送一个小小的消息到目标机器上，判断机器是否可达，其实也就是判断目标机器网络是否能连通。

ping应用的底层，用的是网络层的ICMP协议。

虽然ICMP协议和IP协议都属于网络层协议，但其实ICMP也是利用了IP协议进行消息的传输。

可以理解为ping某个IP就是往某个IP地址发消息

扩展：ICMP报文的介绍。ICMP报文分为两个种类：

1.ICMP差错报告报文，常见的有

- 终点不可达
- 超时
- 参数问题
- 改变路由

2.ICMP询问报文

- 回送请求和回答：向特定主机发出回送请求报文，收到回送请求报文的主机响应回送回答报文
- 时间戳请求和回答：询问对方当前的时间，返回的是一个32位的时间戳

为什么要ping 127.0.0.1?

在ping 127.0.0.1（本地回环地址）时，他发送的消息并不会离开自己的电脑，而是被操作系统内部处理返回。

主要作用：

- 检查本机**TCP/IP**协议栈是否正常：TCP/IP协议是电脑进行通信的基本规则，如果能ping通说明自身通信完好。
- 可以快速定位问题：电脑无法上网时，ping通就是外部网络问题，ping不通就是电脑内部问题
- 本地软件开发与测试：假如你开发了一个服务器，你可以将它与127.0.0.1绑定，这样你就可以通过访问127.0.0.1来测试访问自己的程序，也不需要部署到公网

HTTP和HTTPS

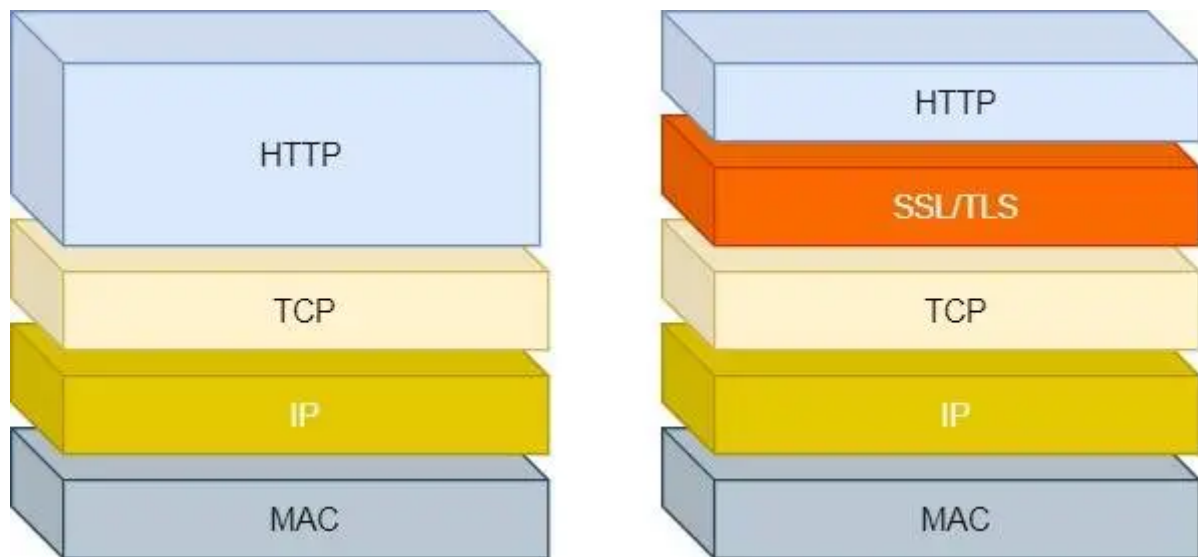
HTTP与HTTPS有哪些区别？

- HTTP是超文本传输协议，信息是明文传递，存在安全风险的问题。HTTPS则解决HTTP不安全的缺陷，在TCP和HTTP网络层之间加入了**SSL/TLS**安全协议，使得报文能够加密传输
- HTTP连接建立相对简单，TCP三次握手之后便可进行HTTP的报文传输。而HTTPS在TCP三次握手之后，还需要进行SSL/TLS的握手过程，才可进入加密报文传输。
- 两者的默认端口不一样，HTTP默认端口号是80，HTTPS默认端口号是443
- HTTPS协议需要向**CA（证书权威机构）**申请数字证书，来保证服务器的身份是可信的。

HTTPS解决了HTTP的哪些问题？

HTTP由于是明文传输，所以安全上存在以下三个风险：

- 窃听风险，比如通信链路上可以获取通信内容，用户信息容易被窃取
- 篡改风险，比如强制植入垃圾广告，视觉污染，用户体验感差
- 冒充风险，比如冒充淘宝网站，用户钱容易没



HTTPS在HTTP与TCP层之间加入了SSL/TLS协议，可以很好的解决了上述的风险：

- 信息加密：交互信息无法被窃取
- 校验机制：无法篡改通信内容，篡改了就不能正常显示
- 身份整数：证明淘宝是真的淘宝网

可见，只要自身不发生问题，SSL/TLS协议是能保证通信是安全的

HTTPS是如何解决上面的三个风险的？

- 混合加密的方式实现信息的机密性，解决了窃听的风险
- 摘要算法的方式来实现完整性，它能够为数据生成独一无二的【指纹】，指纹用于校验数据的完整性，解决了篡改的风险
- 将服务器公钥放入到数字证书中，解决了冒充的风险

1.混合加密



HTTPS采用的是对称加密和非对称加密结合的【混合加密方式】

方式:

- 在通信建立前采用非对称加密的方式交换【会话密钥】，后续就不再使用非对称加密
- 在通信过程中全部使用对称加密的【会话密钥】的方式加密明文数据

采用【混合加密】的方式的原因:

- 对称加密只使用一个密钥，运算速度快，密钥必须保密，无法做到安全的密钥交换
- 非对称加密使用两个密钥：公钥和私钥，公钥可以任意分发而私钥保密，解决了密钥交换问题但速度慢

2.摘要算法+数字签名

为了保证传输的内容不被篡改，我们需要对内容计算出一个【指纹】，然后同内容一起传输给对方。

对方接收到后，也根据内容生成一个【指纹】，两个指纹一经对比，就知道内容被没被篡改过了。

那么，在计算机中会用摘要算法（哈希函数）来计算出内容的哈希值，也就是内容的【指纹】，这个哈希值是唯一的，且不能通过哈希值倒推内容；

但是，通过哈希算法可以确保内容不会被篡改，但是并不能保证【内容+哈希值】不会被中间人替换，因为这里缺少对客户端收到的消息是否来源于服务端的证明。

对于这种情况，我们一般使用非对称加密算法来解决，共有两个密钥：

- 公钥：可以公开给所有人的
- 私钥：必须有本人管理，不能泄露

一般不会用非对称加密来加密实际的传输内容，因为非对称加密的计算比较耗费性能。

私钥是由服务端保管，然后服务端用私钥对客户端颁发公钥，如果客户端收到消息后，能用公钥解密，就说明这个消息是由服务器发送的。

3.数字证书

在前面我们知道

- 可以检验哈希值的一致性，来保证内容没有被篡改
- 可以借助数字签名来确保消息的可靠性（是由持有对应私钥的服务端转发）

还缺少身份验证的关系，如果说这个公钥是伪造的呢？

完全可以伪造出一对公钥和私钥，那么客户端在收到伪造的公钥后，仍然可以用公钥解密伪造私钥加密过后的内容，一样能充当假的服务端。

这就需要CA（数字证书认证机构）来发挥作用了

服务器将公钥注册到CA，CA用自己的私钥对其进行数字签名，然后把【服务器信息+公钥+数字签名】打包成一个数字证书，那么服务器就可以将数字整数发给客户端，然后客户端用已经保存在本地的CA公钥进行解密，来验证数字证书的真实性，如果能解开说明这个是从服务端发来的（可信），然后拿到证书里面的公钥，对发送信息加密后发送给服务端，服务端在用自己的私钥解密。

通过数字证书的方式保证服务器公钥的身份，解决冒充的风险

HTTPS是如何建立连接的？

SSL/TLS协议基本流程：

- 客户端向服务器索要并验证服务器的公钥。
- 双方协商生产【会话密钥】
- 双方采用【会话密钥】进行通信

前两步也就是SSL/TLS的建立过程，也就是TLS握手阶段

TLS的【握手阶段】涉及四次通信，使用不同的密钥交换算法，TLS握手流程也会不一样的，现在常用的密钥交换算法有两种：**RSA**和**ECDHE**

常见的HTTP状态码有哪些？

状态码	类别	含义
1XX	Informational（信息性状态码）	接收的请求正在处理
2XX	Success（成功状态码）	请求正常处理完毕
3XX	Redirection（重定向状态码）	需要进行附加操作以完成请求
4XX	Client Error（客户端错误状态码）	服务器无法处理请求
5XX	Server Error（服务器错误状态码）	服务器处理请求出

1XX信息

100 continue: 表明到目前为止都很正常，客户端可以继续发送请求或者忽略这个响应

2XX成功

200 OK

204 No Content: 请求已经成功处理，但是返回的响应报文不包含实体的主体部分。一般在只需要从客户端往服务器发送信息，而不需要返回数据的时候使用

206 Partial Content: 表示客户端进行了范围请求，响应报文包含由**Content-Range**指定范围的实体内容

3XX重定向

301 Moved Permanently: 永久性重定向

302 Found: 临时性重定向

303 See Other: 和302有着相同的功能，但是303明确要求客户端应该采用GET方法获取资源

4XX客户端错误

400 Bad Request: 请求报文中存在语法错误

403 Forbidden: 请求被拒绝

404 Not Found: 没有找到服务器

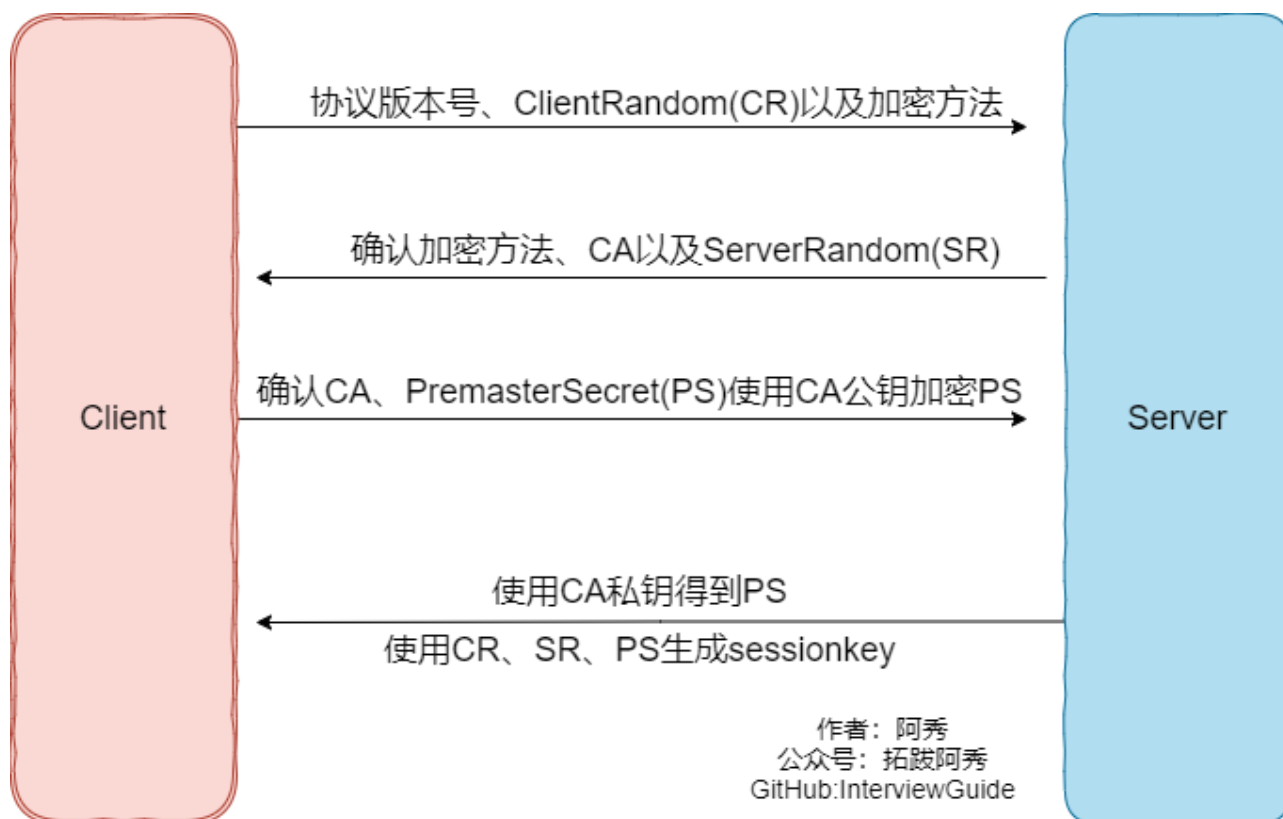
5XX

500 Internal Server error: 服务器正在执行请求时发生错误

503 Service Unavailable: 服务器暂时处于超负载或正在停机维护，现在无法处理请求

HTTPS采用的加密方式有哪些？对称or非对称？

HTTPS采用混合的加密机制，使用非对称密钥加密用于传输对称密钥来保证传输过程的安全性，之后使用对称密钥加密进行通信来保证通信过程的效率



确保传输安全过程（就是RSA原理）：

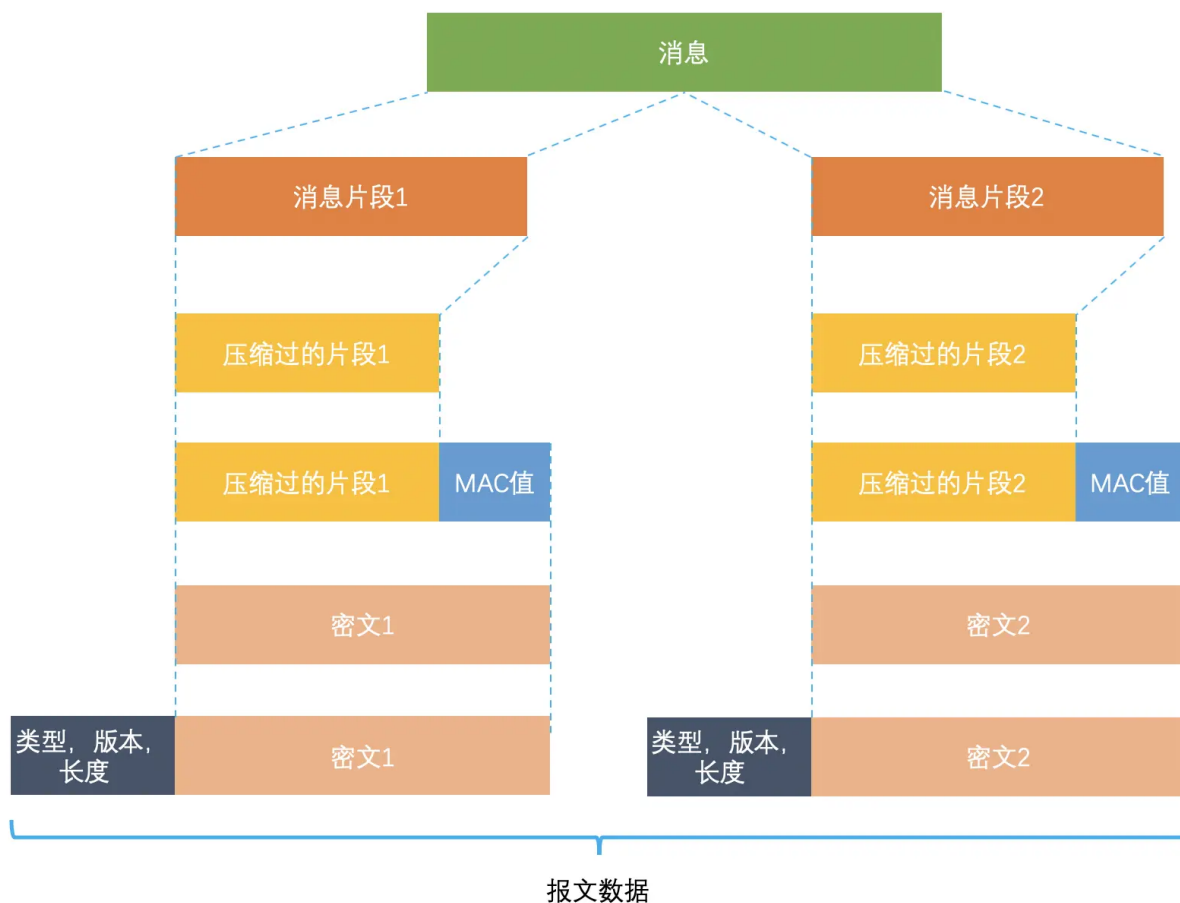
- 1.client给出协议版本号、一个客户端生成的随机数（client random），以及客户端支持的加密方法
- 2.Server确认双方使用的加密方法，并给出数字证书、以及一个服务器生成的随机数（Server random）
- 3.client确认数字证书有效，然后生成一个新的随机数（premaster secret），并使用数字证书中的公钥，加秘这个随机数，发给server
- 4.server使用自己的私钥，获取client发来的随机数
- 5.client和server根据约定的加密方法，使用前面的三个随机数，生成“对话密钥”，用来加密接下来的整个对话过程

HTTPS的应用数据是如何保证完整性的？

TLS在实现上分为握手协议和记录协议两层：

- **TLS**握手协议就是我们前面说的TLS四次握手过程，负责协商加密算法和生成对称密钥，后续用此密钥来保护应用程序数据。
- **TLS**记录协议负责保护应用程序数据并验证其完整性和来源，所以对HTTP数据加密是使用记录协议

TLS记录协议主要负责消息（HTTP数据）的压缩，加密及数据的认证，



具体过程如下：

- 首先，消息被分割成多个较短的片段，然后分别对每个片段进行压缩
- 接下来，经过压缩的片段会被加上消息认证码（MAC值，这个是通过哈希算法生成的），这是为了保证完整性，并进行数据的认证。通过附加消息认证码的MAC值，可以识别出篡改。于此同时，为了防止重放攻击，在计算消息认证码时，还加上了片段的编码。
- 再接下来，经过压缩的片段再加上消息认证码会一起通过对称密码进行加密
- 最后，上述经过加密的数据再加上由数据类型、版本号，压缩后的长度组成的报头就是最终的报文数据

记录协议完成后，最终的报文数据将传递到传输控制协议 (TCP) 层进行传输。

HTTPS一定安全可靠吗？

HTTPS主要解决了数据传输中的三大威胁：窃听、篡改和冒充，但它仍然无法解决如下问题：

- 服务端不安全：如果网站服务器本身存在漏洞或被黑客入侵，即使数据通过HTTPS安全送达，也会被从服务器上窃取
- 客户端设备不安全：如果你的电脑或手机感染了木马、病毒或者恶意软件，它们可以在数据进入HTTPS加密通道之前就将其窃取
- 钓鱼网站：钓鱼网站同样可以申请并部署有效的HTTPS证书。当访问钓鱼网站时，浏览器地址栏会显示安全锁标志，让你误以为很安全
- 用户自身行为：使用弱密码、在不同网站重复使用密码、点击不明链接等行为，都是HTTPS无法保护的
- 用户主动授权软件：在使用某些抓包工具并授权其进行网络监控时，就等于创建了一个中间人，此时HTTPS的保护也会失效

完整的HTTP请求过程包括哪些内容

第一种回答

- 建立起客户机和服务器之间的连接
- 建立连接后，客户机发送一个请求给服务器
- 服务器收到请求给予响应信息
- 客户端浏览器将返回的内容解析并呈现，断开连接

第二种回答

域名解析 -> 发起TCP的三次握手 -> 建立TCP连接后发起http请求 -> 服务器响应http请求，浏览器得到html代码 -> 浏览器解析html代码，并请求html代码中的资源（如js、css、图片等） -> 浏览器对页面进行渲染呈现给用户

什么是DNS

DNS（域名系统），因特网上作为域名和IP地址互相映射的一个分布式数据库，能够使用户更方便的访问互联网，而不用去记住能够被机器直接读取的IP数串。

通过主机名，最终得到该主机名对应的IP地址的过程叫做域名解析

DNS为应用层协议

为什么域名解析用UDP协议

UDP的传输速度非常快，UDP的DNS协议只要一个请求、一个应答就好了

而使用基于TCP的DNS协议要三次握手、发送数据以及应答、四次挥手，但是UDP协议传输内容不能超过512字节

不过客户端向DNS服务器查询域名，一般返回的内容都不超过512字节，用UDP传输即可。

GET和POST的区别

1.get是获取数据，post是修改数据

2.get把请求放在url上，以?分割URL和传输数据，参数之间以&相连，所以get不太安全。而post把数据放在HTTP的包体内（request body 相对安全）

3.get提交的数据最大是2k（限制实际上取决于浏览器），post理论上没有限制

4.get产生一个TCP数据包，浏览器会把http header和data一并发送出去，服务器响应200（返回数据）；post产生两个TCP数据包，浏览器先发送header，服务器响应100continue，浏览器再发送data，服务器响应200 ok（返回数据）

5.get请求会被浏览器主动缓存，而post不会，除非手动设置

6.本质区别：get是幂等的，而post不是幂等的

正因为它们有这样的区别，所以不应该且不能用get请求做数据的增删改这些有副作用的操作。因为get请求是幂等的，在网络不好的隧道中会尝试重试。如果用get请求增数据，会有重复操作的风险，而这种重复操作可能会导致副作用（浏览器和操作系统并不知道你会用get请求去做增操作）

DNS负载均衡是什么策略？

当一个网络有足够多的用户的时候，加入每次请求的资源都位于同一台机器上面，那么这台机器随时可能会崩掉。

处理办法就是用DNS负载均衡技术，它的原理是在DNS服务器中为同一个主机名配置多个IP地址，在应答DNS查询时，DNS服务器对每个查询将以DNS文件中主机记录的IP地址按顺序返回不同的解析结果，将客户端的访问引导到不同的机器上去，使得不同的客户端访问不同的服务器，从而达到负载均衡的目的。例如，可以根据每台机器的负载量，该机器离用户地理位置的距离等等。

什么是SSL/TLS?

SSL代表安全套接字层。它是一种用于加密和验证应用程序（如浏览器）和Web服务器之间发送的数据的协议。身份验证，加密https的加密机制是一种共享密钥加密和公开密钥加密并用的混合加密机制。

SSL/TLS协议作用：认证用户和服务，加密数据，维护数据的完整性的应用层协议加密和解密需要两个不同的密钥，所以被称为非对称加密；加密和解密都使用同一个密钥的对称加密：优点在于加密、解密的效率通常比较高，https是基于非对称加密的，公钥是公开的

为什么有时候刷新页面不需要重新建立SSL连接？

TCP连接有的时候会被浏览器和服务端维持一段时间，TCP不需要重新建立，SSL自然也会用之前的

SSL中的认证中的证书是什么？

通过使用证书来对通信方进行认证

数字证书认证机构（CA），是客户端与服务器双方都可信赖的第三方机构。

服务器的运营人员向CA提出公开密钥的申请，CA在判明提出申请者的身份后，会对已申请的公开密钥做数字签名，然后分配这个已签名的公开密钥，并将该公开密钥放入公开密钥证书后绑定在一起。

进行HTTPS通信时，服务器会把证书发给客户端，客户端取得其中的公开密钥之后，先使用数字签名进行验证，如果验证通过，就可以开始通信了

HTTPS如何保证数据传输安全，整体流程？

- （1）客户端向服务端发起SSL连接请求
- （2）服务器把公钥发给客户端，并且服务器端保存着唯一的私钥
- （3）客户端用公钥对双方通信的对称密钥进行加密，并发送给服务器端
- （4）服务器利用自己唯一的私钥对客户端发来的对称密码进行解密
- （5）进行数据传输，服务器和客户端双方用公有的相同的对称密钥对数据进行加解密，可以保证在数据收发过程中的安全，即是第三方获得数据包，也无法对其进行加密，解密和篡改

因为数字签名、摘要是证书防伪非常关键的武器。“摘要”就是对传输的内容，通过hash算法计算出一段固定长度的串。然后，通过发送方的私钥对这段摘要进行加密，加密后得到的结果就是“数字签名”

SSL/TLS协议的基本思路是采用公钥加密法，也就是说，客户端先向服务器索要公钥，然后用公钥加密信息，服务器收到密文后，用自己的私钥解密

如何保证公钥不被篡改

将公钥放在数字证书中。只要证书是可信的，公钥就是可信的。

HTTP请求和响应报文有哪些主要字段？

请求报文：

简单来说：

- 请求行：Request Line
- 请求头：Request Headers
- 请求体：Request Body

响应报文：

简单来说：

- 状态行：Status Line
- 响应头：Response Headers
- 响应体：Response Body

Cookie是什么？

HTTP协议是无状态的，主要是为了让HTTP协议尽可能简单，使得它能够处理大量事务，HTTP/1.1 引入Cookie来保存状态信息。

Cookie是服务器发送到用户浏览器并保存在本地的一小块数据，它会在浏览器之后向同一服务器再次发起请求时被携带上，用于告知服务端两个请求是否来自同一浏览器。由于之后每次请求都会需要携带Cookie数据，因此会带来额外的性能开销。

Cookie曾一度用于客户端数据的存储，因为当时并没有其他合适的存储办法而作为唯一的存储手段，但现在随着现代浏览器开始支持各种各样的存储方式，**Cookie**渐渐被淘汰。

新的浏览器API已经允许开发者直接将数据存储到本地，如使用**Web storage API**（本地存储和会话存储）或**IndexedDB**。

cookie的出现是因为**HTTP**是无状态的一种协议，换句话说，服务器记不住你，可能你每刷新一次网页，就要重新输入一次账号密码进行登录。这显然无法接受，**cookie**的作用就好比服务器给你贴个标签，然后你每次向服务器再发起请求时，服务器就能够根据**cookie**认出你。

Cookie有什么用途？

1. 会话状态管理（如用户登录状态、购物车、游戏分数）
2. 个性化设置（如用户自定义设置、主题等）
3. 浏览器行为跟踪（如跟踪分析用户行为等）

Session是什么？

除了可以将用户信息通过**Cookie**存储在用户的浏览器中，也可以利用**Session**存储在服务器端，存储在服务器端的信息更加安全。

Session 可以存储在服务器上的文件、数据库或者内存中。也可以将**Session**存储在**Redis**这种内存型数据库中，效率会更高。

使用**Session**维护用户登录状态的过程如下：

1. 用户进行登陆时，用户提交包含用户名和密码的表单，放入**HTTP**请求报文中；
2. 服务器验证该用户名和密码，如果正确则把用户信息存储到**Redis**中，它在**Redis**中的Key称为**Session ID**；
3. 服务器返回的响应报文的**Set-Cookie**首部字段包含了这个**Session ID**，客户端收到响应报文后将该**Cookie**值存入浏览器中；
4. 客户端之后对同一个服务器进行请求时会包含该**Cookie**值，服务器收到之后取出**Session ID**，从**Redis**中取出用户信息，继续之前的业务操作

使用Session的过程是怎样的？

过程如下：

用户进行登录时，用户提交包含用户名和密码的表单，放入HTTP请求报文中

服务器验证该用户名和密码，如果正确则把用户信息存储到Redis中，他在Redis中的key称为Session ID

服务器返回的响应报文的Set-Cookie首部字段包含了这个Session ID,客户端收到响应报文之后将该Cookie值存入浏览器中

客户端之后对同一个服务器进行请求时会包含该Cookie值，服务器收到之后提取出Session ID，从Redis中取出用户信息，继续之前的业务操作

注意：Session ID的安全性问题，不能让他被恶意攻击者轻易获取，那么就不能产生一个容易被猜到的Session ID值。此外，还需要经常重新生成Session ID。在对安全性要求极高的场景下，例如转账等操作，除了使用Session管理用户状态之外，还需要对用户进行重新验证，比如重新输入密码，或者使用短信验证码等方式

Session和Cookie应该如何选择（使用场景）？

- Cookie只能存储ASCII码字符串，而Session则可以存储任何类型的数据，因此在考虑数据复杂性时首选Session
- Cookie存储在浏览器中，容易被恶意查看。如果非要将一些隐私数据存在Cookie中，可以将Cookie值进行加密，然后在服务器进行解密
- 对于大型网站，如果用户所有的信息都存储在Session中，那么开销是非常大的，因此不建议将所有的用户信息都存储到Session中

Cookies和Session区别是什么？

Cookie和Session都是客户端与服务器之间保持状态的解决方案

1.存储的位置不同

cookie: 存放在客户端

session: 存放在服务端。Session存储的数据比较安全

2.存储的数据类型不同

两者都是key-value的结构，但针对value的类型是有差异的

cookie: value只能是字符串类型，**session:** value是任意类型的对象（Object、List等）

3.存储的数据大小限制不同

cookie: 大小受浏览器的限制，很多是4K的大小

session: 理论上受当前内存的限制

4.生命周期的控制

cookie的生命周期当浏览器关闭的时候，就消亡了

（1）**cookie**的生命周期是累计的，从创建时，就开始计时，可以自定义

（2）**session**的生命周期是间隔的，从创建时，开始计时如在20分钟，没有访问**session**，那么**session**生命周期被销毁，默认依赖浏览器会话（关闭浏览器即失效），也可以自定义

虽然它们有区别，但在实际开发中，**Session**往往需要依赖 **Cookie** 才能实现。其工作流程如下：

1. 用户登录：用户首次向服务器发起请求并登录成功。
2. 生成 **Session**：服务器在内存中创建一个 **Session** 对象，用来存储该用户的登录状态（如 `user_id=1001`），并生成一个唯一的 **Session ID**。
3. 下发 **Cookie**：服务器将这个 **Session ID** 通过响应头（`Set-Cookie`）发送给浏览器，浏览器将其保存为 **Cookie**。
4. 后续请求：用户再次访问网站时，浏览器会自动在请求头中带上这个包含 **Session ID** 的 **Cookie**。
5. 身份校验：服务器收到请求，解析出 **Cookie** 中的 **Session ID**，去内存中查找对应的 **Session** 数据，从而确认“你是谁”。

MTU和MSS分别是什么？

MTU（maximum transmission unit）：最大传输单元，由硬件规定，如以太网的MTU为1500字节。

MSS（maximum segment size）：最大分节大小，为TCP数据包每次传输的最大数据分段大小，一般由发送端向对端TCP通知对端在每个分节中能发送的最大TCP数据。MSS值为MTU减去IPV4 Header（20 bytes）和TCP Header（20 bytes）得到

什么是SQL注入攻击？

攻击者在HTTP请求中注入恶意的SQL代码，服务器使用参数构建数据库SQL命令时，恶意SQL被一起构造，并在数据库中执行。

用户登录，输入用户名...，密码'or '1' = 1，如果此时使用参数构造的方式，就会出现select * from user where name = 'lianggzone' and password = " or '1' = '1' 不管用户名和密码是什么内容，使查询出来的用户列表不为空。如何防范SQL注入攻击使用预编译的PreparedStatement是必须的，但是一般我们会从两个方面同时入手。

Web端

- （1）有效性检验
- （2）限制字符串输入的长度

服务端

- （1）不用拼接SQL字符串
- （2）使用预编译的PreparedStatement
- （3）有效性检验。（为什么服务端还要做有效性检验？第一准则，外部都是不可信的，防止攻击者绕过Web端请求）
- （4）过滤SQL需要的参数中的特殊字符。比如单引号、双引号

HTTP中缓存的私有和共有字段？

private指令规定了将资源作为私有缓存，只能被单独用户使用，一般存储在用户浏览器中。

```
Cache-Control: private
```

public指令规定了将资源作为公共缓存，可以被多个用户使用，一般存储在代理服务器中。

```
Cache-Control: public
```

GET方法参数写法是固定的吗？

在约定中，我们的参数是写在？后面，用&分割。

我们知道，解析报文的过程是通过获取TCP数据，用正则等工具从数据中获取Header和Body，从而提取参数

比如header请求头中添加token，来验证用户是否登录等权限问题

也就是说，我们可以自己约定参数的写法，只要服务端能够解释出来就行

GET方法的长度限制是怎么回事？

网络上都会提到浏览器地址栏输入的参数是有限的

首先说明一点，HTTP协议没有Body和URL的长度限制，对URL限制的大多是浏览器和服务器的原因

浏览器就不说了，服务器是因为处理长URL要消耗比较多的资源，为了性能和安全（防止恶意构造长URL来攻击）考虑，会给URL长度加限制

HTTP中有个缓存机制，但如何保证缓存是最新的？

max-age指令出现在请求报文，并且缓存资源的缓存时间小于该指令指定的时间，那么就能接收该缓存

max-age指令出现在响应报文，表示缓存资源在缓存服务器中保存的时间

```
Cache-Control: max-age=3153600
```

Expires首部字段也可以用于告诉缓存服务器该资源什么时候会过期。

```
Expires: . . .
```

- 在HTTP/1.1，会优先处理max-age指令
- 在HTTP/1.0，max-age指令会被忽略掉

浏览器在与服务器建立了一个TCP连接后是否会在一个HTTP请求完成后断开？什么情况下会断开？

在HTTP/1.0中，一个服务器在发送完一个HTTP响应后，会断开TCP链接。但是这样每次请求都会重新建立和断开TCP连接，代价过大。所以虽然标准中没有设定，某些服务器对Connection: keep-alive的Header进行了支持。

意思是说，完成这个HTTP请求之后，不要断开HTTP请求使用的TCP连接。这样的好处是连接可以被重新使用，之后发送HTTP请求的时候不需要重新建立TCP连接，以及如果维持连接，那么SSL的开销也可以避免。

持久连接：既然维持TCP连接好处这么多，HTTP/1.1就把Connection头写进标准，并且默认开启持久连接，除非请求中写明Connection: close，那么浏览器和服务器之间会维持一段时间的TCP连接，不会一个请求结束就断掉

默认情况下TCP连接不会断开，只有在请求报头中声明Connection: close才会在请求完成后关闭连接

HTTP/1.1、HTTP/2、HTTP/3演变

HTTP/1.1相比HTTP/1.0提高了什么性能？

性能上的改进：

- 使用长连接的方式改善了HTTP/1.0短链接造成的性能开销
- 支持管道（pipeline）网络传输，只要第一个请求发出去了，不必等其回来，就可以发第二个请求出去，可以减少整体的响应时间

但HTTP/1.1还是有性能瓶颈：

- 请求/响应头部（Header）未经压缩就发送，首部信息越多延迟越大。只能压缩Body的部分
- 发送冗长的首部。每次互相发送相同的首部造成的浪费较多
- 服务器是按请求的顺序响应的，如果服务器响应慢，会招致客户端一直请求不到数据，也就是队头阻塞
- 没有请求优先级控制
- 请求只能从客户端开始，服务器只能被动响应

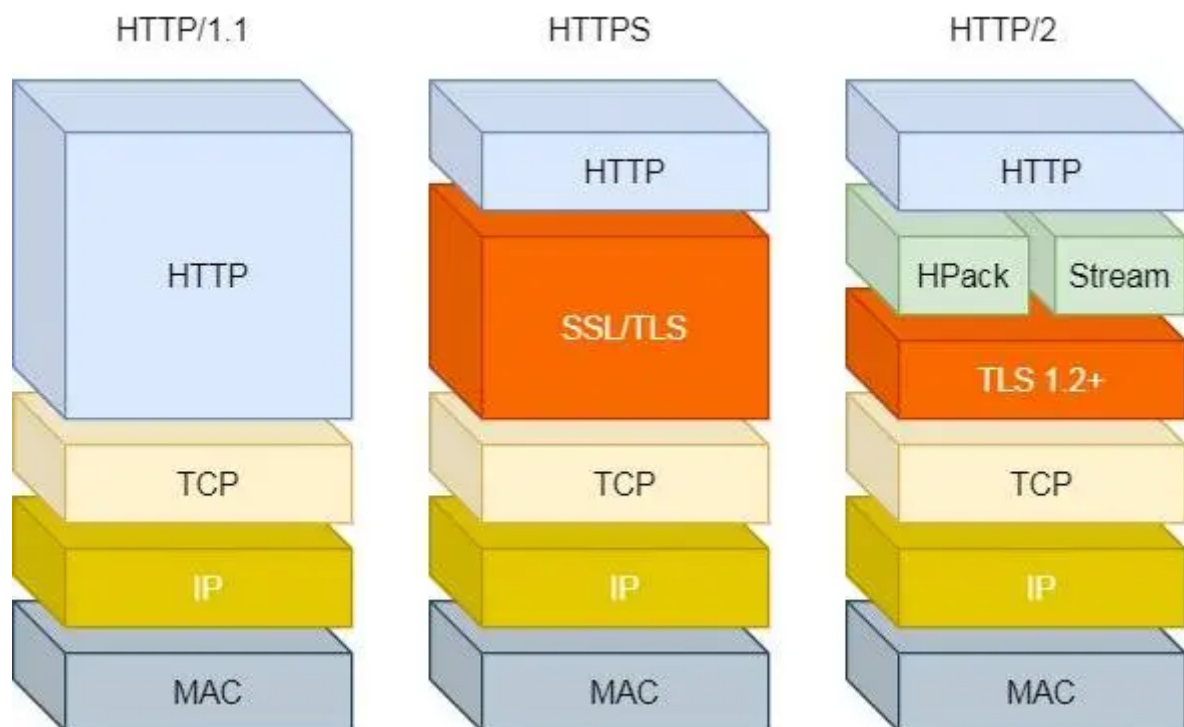
什么时候HTTP支持断点续传？实现原理？

HTTP协议从1.1版本开始正式支持断点续传。在HTTP/1.0时代，如果文件传输从中途断开，客户端只能从头重新下载；而HTTP/1.1引入了“字节范围请求”机制，极大地提升了文件传输的可靠性和效率，实现机制如下：

- 1.核心原理：客户端通过Range请求头指定需要下载的字节范围；服务器返回206（Partial Content）状态码及对应的局部数据
- 2.实现流程：客户端记录已下载的文件偏移量，断线恢复后从断点处发起续传请求，将新数据追加到本地文件
- 3.防损坏机制：客户端在续传请求头中加入If-Range（携带 ETag 或 Last-Modified）。若服务器文件未变则返回206续传；若文件已更新则返回200全量重传，避免文件拼接损坏

HTTP/2做了什么优化？

HTTP/2协议是基于HTTPS的，所以HTTP/2的安全性也是有保障的。



HTTP/2相比于HTTP/1.1性能上的改进：

- 头部压缩
- 二进制格式
- 并发传输
- 服务器主动推送资源

1.头部压缩

HTTP/2 会压缩头（**header**）如果你同时发出多个请求，他们的头是一样的或是相似的，那么，协议会帮你消除重复的部分。

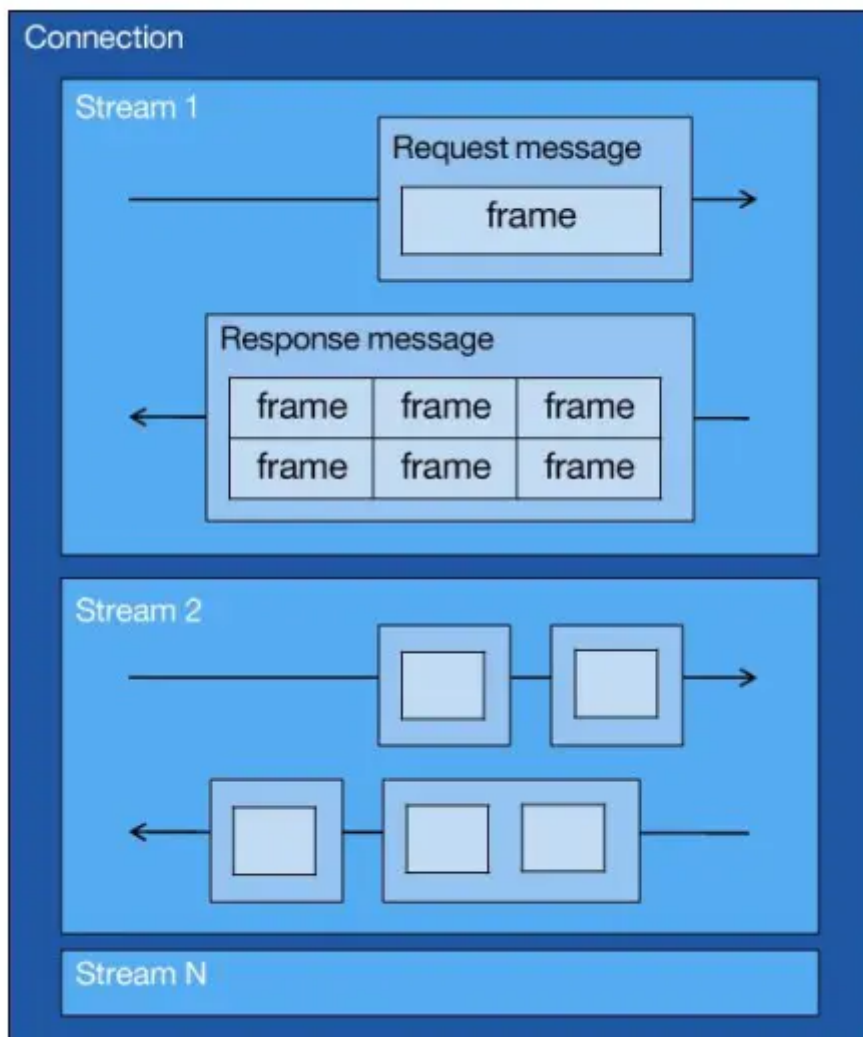
这就是所谓的HPACK算法；在客户端和服务端同时维护一张头信息表，所有字段都会存入这个表，生成一个索引号，以后就不发送同样字段了，只发送索引号，这样就提高速度了

2.二进制格式

HTTP/2不再像HTTP/1.1里的纯文本形式的报文，而是全面采用了二进制格式，头信息和数据体都是二进制，并且统称为帧（**frame**）：头信息帧和数据帧

3.并发传输

在HTTP/1.1中是基于请求-响应模型的，HTTP必须在等待这一个事务（请求与响应）处理完成后，



上图可以看到，1 个 TCP 连接包含多个 Stream，Stream 里可以包含 1 个或多个 Message，Message 对应 HTTP/1 中的请求或响应，由 HTTP 头部和包体构成。Message 里包含一条或者多个 Frame，Frame 是 HTTP/2 最小单位，以二进制压缩格式存放 HTTP/1 中的内容（头部和包体）。

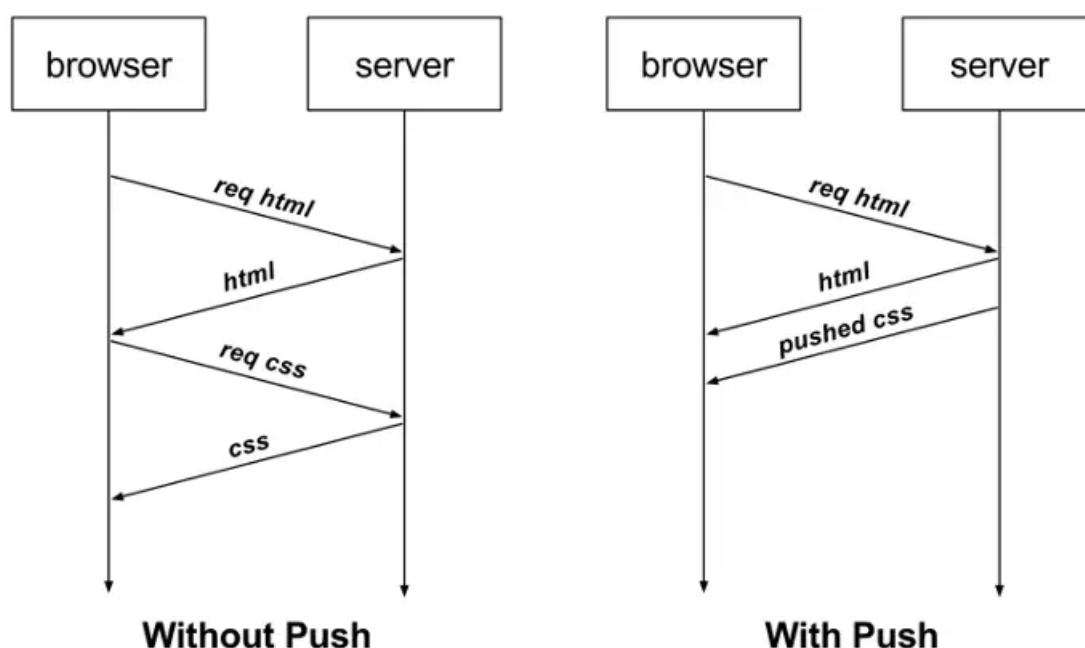
针对不同的 HTTP 请求用独一无二的 Stream ID 来区分，接收端可以通过 Stream ID 有序组装成 HTTP 消息，不同 Stream 的帧是可以乱序发送的，因此可以并发不同的 Stream，也就是 HTTP/2 可以并行交错地发送请求和响应。

4.服务器推送

HTTP/2还在一定程度上改善了传统的【请求-应答】工作模式。服务端不再是被动地响应，可以主动向客户端发送消息。

客户端和服务端双方都可以建立 Stream，Stream ID 也是有区别的，客户端建立的 Stream 必须是奇数号，而服务器建立的 Stream 必须是偶数号。

在HTTP/1.1中客户端如果想服务器请求HTML文件，收到后，可能还需要向服务器请求CSS文件来渲染页面，所以需要两次文件请求，也就是两次信息往返。

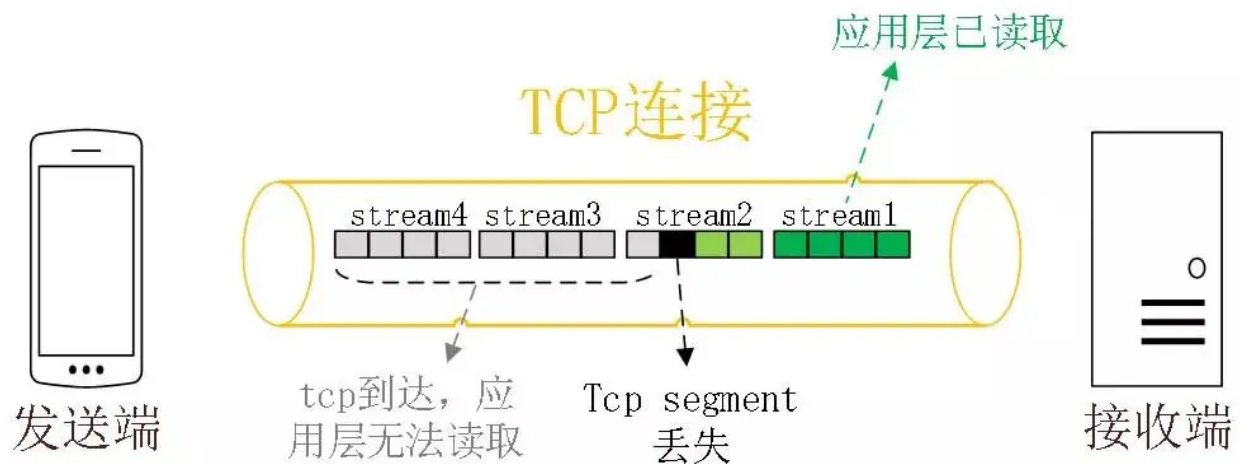


但是在HTTP/2中，客户端在访问HTML时，服务器可以直接主动推送CSS文件，减少了消息传递的次数

HTTP/2有什么缺陷？

HTTP/2通过Stream的并发能力，解决了HTTP/1队头阻塞的问题，看似很完美了，但是HTTP/2还是存在“队头阻塞”的问题，只不过问题不是在HTTP这一层面，而是在TCP这一层

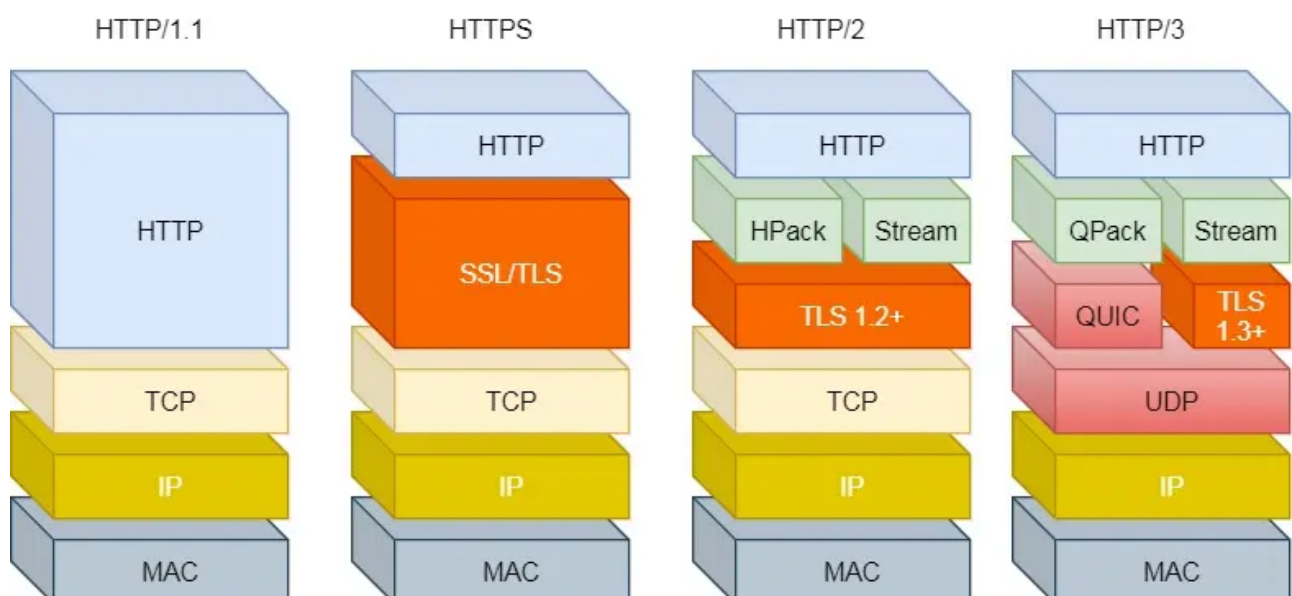
HTTP/2是基于TCP协议来传输数据的，TCP是字节流协议，TCP层必须保证收到的字节数据是完整且连续的，这样内核才会将缓冲区里的数据返回给HTTP应用，那么当【前一个字节数据】没有到达时，后收到的字节数据只能存放在内核缓冲区里，只有等到这1个字节数据到达时，HTTP/2应用层才能从内核中拿到数据，这就是HTTP/2队头阻塞问题



所以一旦发生了丢包现象，就会触发TCP的重传机制，这样在一个TCP连接中的所有HTTP请求都必须等待这个丢了包重传回来。

HTTP/3做了哪些优化？

HTTP/2队头阻塞的问题是因为TCP，所以HTTP/3把HTTP下层的TCP协议改成了UDP协议



UDP 发送是不管顺序，也不管丢包的，所以不会出现像 HTTP/2 队头阻塞的问题。大家都知道 UDP 是不可靠传输的，但基于 UDP 的 QUIC 协议可以实现类似 TCP 的可靠性传输。

QUIC协议的三个特点

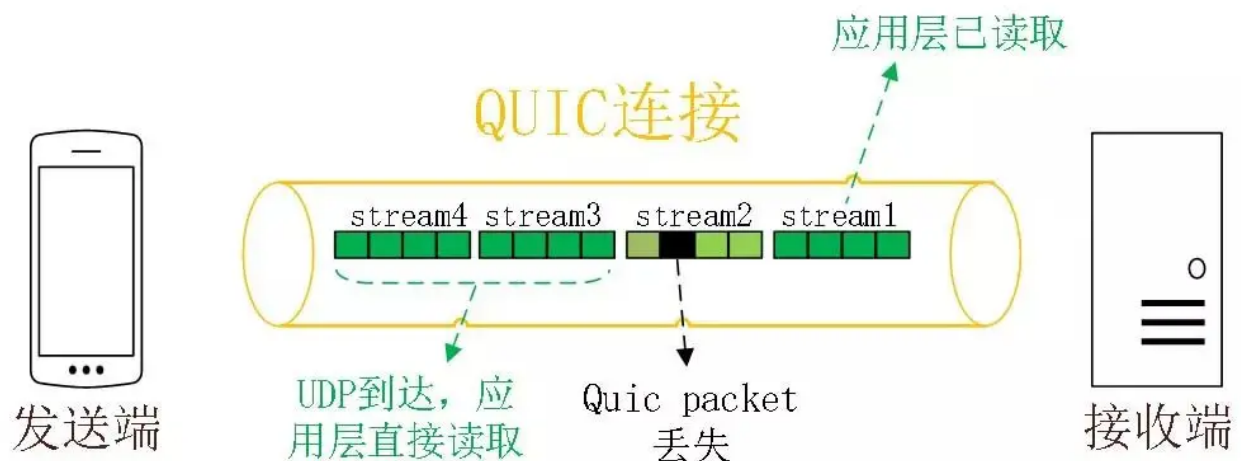
- 无队头阻塞
- 更快的连接建立
- 连接迁移

1.无队头阻塞

QUIC 协议也有类似 HTTP/2 Stream 与多路复用的概念，也是可以在同一条连接上并发传输多个 Stream，Stream 可以认为就是一条 HTTP 请求。

QUIC 有自己的一套机制可以保证传输的可靠性的。当某个流发生丢包时，只会阻塞这个流，其他流不会受到影响，因此不存在队头阻塞问题。这与 HTTP/2 不同，HTTP/2 只要某个流中的数据包丢失了，其他流也会因此受影响。

所以，QUIC 连接上的多个 Stream 之间并没有依赖，都是独立的，某个流发生丢包了，只会影响该流，其他流不受影响。

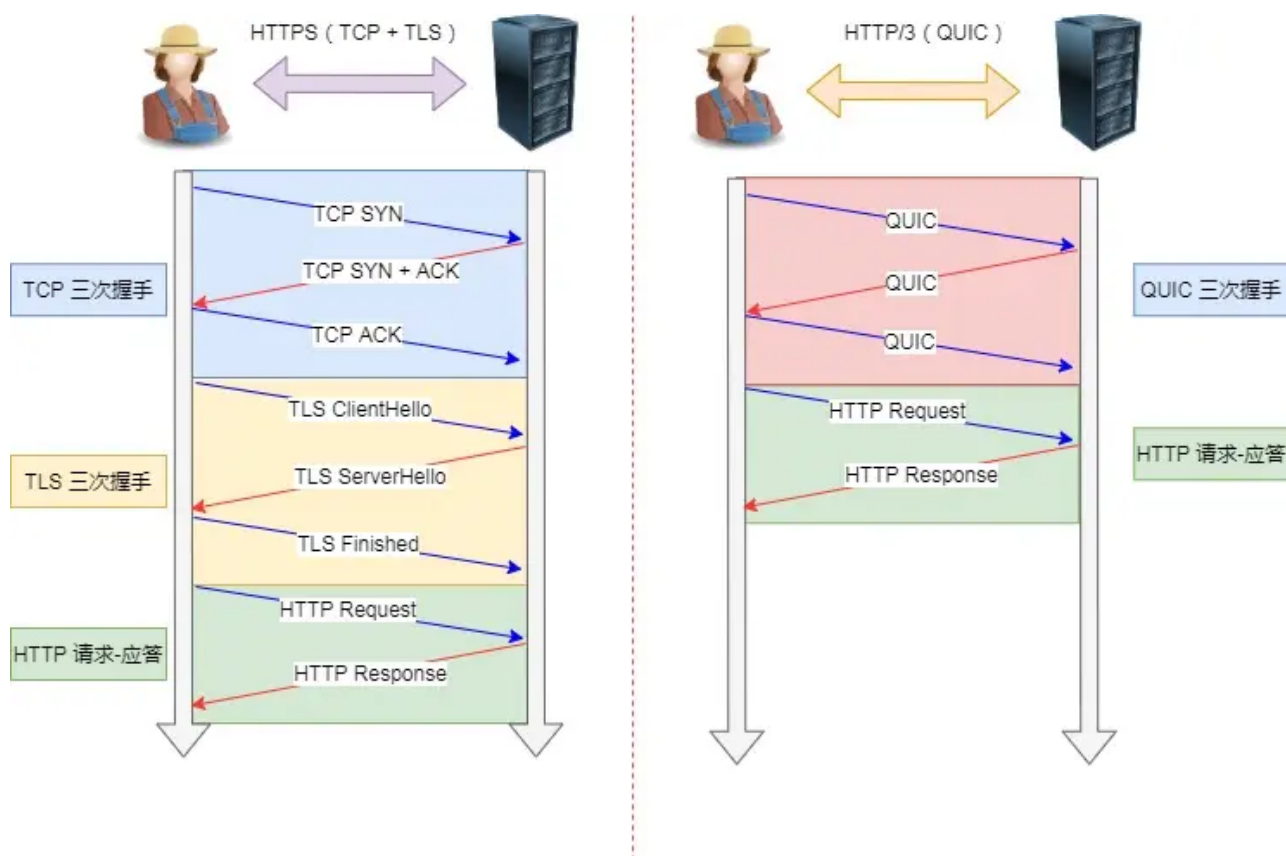


2.更快的连接建立

对于 HTTP/1 和 HTTP/2 协议，TCP 和 TLS 是分层的，分别属于内核实现的传输层、openssl 库实现的表示层，因此它们难以合并在一起，需要分批次来握手，先 TCP 握手，再 TLS 握手。

HTTP/3 在传输数据前虽然需要 QUIC 协议握手，但是这个握手过程只需要 1 RTT，握手的目的是为确认双方的「连接 ID」，连接迁移就是基于连接 ID 实现的。

但是 HTTP/3 的 QUIC 协议并不是与 TLS 分层，而是 QUIC 内部包含了 TLS，它在自己的帧会携带 TLS 里的“记录”，再加上 QUIC 使用的是 TLS/1.3，因此仅需 1 个 RTT 就可以「同时」完成建立连接与密钥协商，如下图：



3. 连接迁移

基于TCP传输协议的HTTP协议，由于是通过四元组（源IP、源端口、目的IP、目的端口）确定一条TCP连接。



那么当移动设备的网络从 4G 切换到 WIFI 时，意味着 IP 地址变化了，那么就必须要断开连接，然后重新建立连接。而建立连接的过程包含 TCP 三次握手和 TLS 四次握手的时延，以及 TCP 慢启动的减速过程，给用户的感觉就是网络突然卡顿了一下，因此连接的迁移成本是很高的。

而 QUIC 协议没有用四元组的方式来“绑定”连接，而是通过连接 ID 来标记通信的两个端点，客户端和服务端可以各自选择一组 ID 来标记自己，因此即使移动设备的网络变化后，导致 IP 地址变化了，只要仍保有上下文信息（比如连接 ID、TLS 密钥等），就可以“无缝”地复用原连接，消除重连的成本，没有丝毫卡顿感，达到了连接迁移的功能。

所以，QUIC 是一个在 UDP 之上的伪 TCP + TLS + HTTP/2 的多路复用的协议。

为什么需要基于 UDP 实现可靠传输？

TCP 天然支持可靠传输，但存在四个痛点：

1. 升级困难：TCP 在操作系统内核中实现，迭代周期极长。
2. 连接建立延迟高：TCP 握手 + TLS 握手需要多次 RTT。
3. 队头阻塞：TCP 必须按序确认，一个包丢了后面全堵住。
4. 网络迁移要重连：TCP 用四元组（源IP、源端口、目的IP、目的端口）标识连接，IP 一变就得断开重建。

QUIC 就是基于 UDP 在应用层重新实现可靠传输，逐一解决这些问题。目前已应用于 HTTP/3。

QUIC 的协议头部结构

QUIC 在 UDP 报文头和 HTTP 消息之间加了三层头部：Packet Header、QUIC Frame Header、然后才是 HTTP 消息。

1. Packet Header

分为两种：

- **Long Packet Header**：首次建立连接时使用，用于协商连接 ID。
- **Short Packet Header**：日常传输数据时使用，不需要再传 Source Connection ID，只传 Destination Connection ID。

关键设计：**Packet Number** 严格递增。即使重传的报文，**Packet Number** 也是递增的，不会复用原来的编号。

这解决了 TCP 的重传歧义问题：TCP 重传报文和原始报文序列号相同，收到 ACK 时无法区分是原始报文的响应还是重传报文的响应，导致 RTT（往返时间）计算不精确。QUIC 的 **Packet Number** 单调递增，ACK 带回的编号能明确对应是哪个报文，RTT 计算无歧义。

另外，单调递增的 **Packet Number** 还让 QUIC 支持乱序确认：丢包后窗口仍可继续滑动，不会卡在原地。

2. QUIC Frame Header

一个 **Packet** 中可以放多个 **Frame**。以 **Stream** 类型为例，关键字段：

- **Stream ID**: 区分并发的多条 HTTP 请求。
- **Offset**: 类似 TCP 的 Seq 序号，保证数据的顺序性和可靠性。
- **Length**: 指明 **Frame** 数据长度。

重传报文的 **Packet Number** 变了，怎么确定内容和原来一样？通过 **Stream ID + Offset** 判断。如果两个数据包的 **Stream ID** 和 **Offset** 一致，内容就一致。接收端据此按顺序组装数据交给应用层。

总结：QUIC 通过单向递增的 **Packet Number** 配合 **Stream ID + Offset**，实现了乱序确认且不影响数据正确组装，摆脱了 TCP 必须按序确认的限制。

QUIC 如何解决队头阻塞

TCP 的队头阻塞

TCP 接收窗口必须收到有序数据才能滑动。比如窗口范围是 3251 字节，如果第 32 字节没收到，即使 3340 字节先到了也无法被应用层读取，窗口不滑动，后续数据全部阻塞。

HTTP/2 的队头阻塞

HTTP/2 引入了 **Stream** 概念实现并发传输，不同 **Stream** 的帧可以乱序发送。但所有 **Stream** 跑在同一条 TCP 连接上，共享同一个 TCP 滑动窗口。某个 **Stream** 丢包导致 TCP 窗口不滑动，所有 **Stream** 都被阻塞。

QUIC 的解决方案

QUIC 也用 **Stream** 概念，但每个 **Stream** 有独立的滑动窗口。**Stream** 之间互不依赖，**Stream 2** 丢了包只影响 **Stream 2** 自己，不影响 **Stream 1**、**Stream 3** 的传输和读取。

QUIC 的流量控制

QUIC 实现了两个级别的流量控制：

1. Stream 级别

每个 **Stream** 有独立的滑动窗口。接收窗口左边界取决于已接收到的最大偏移字节数（不像 TCP 必须等前面所有字节都收到才滑动）。当已接收的有序数据超过最大接收窗口的一半时，窗口右边界向右扩展，同时发送窗口更新帧通知对端。

2. Connection 级别

限制连接中所有 **Stream** 的总字节数，防止发送方超过连接的缓冲容量。**Connection** 可用窗口 = 各 **Stream** 可用窗口之和。

QUIC 对拥塞控制的改进

QUIC 默认使用 TCP 的 **Cubic** 拥塞控制算法（慢开始、拥塞避免、快重传、快恢复），同时支持 **BBR**、**Reno** 等算法。

改进点：

- 应用层实现：不需要操作系统内核支持，随浏览器更新即可迭代，升级速度快。
- 灵活选择：可以为不同应用设置不同的拥塞控制算法，不像 TCP 改算法是对系统所有应用全局生效。

QUIC 更快的连接建立

- **HTTP/1.1**：TCP 握手 1RTT + TLS 握手 2RTT = 共 3 RTT。
- **HTTP/2**（会话复用）：TCP 1RTT + TLS 1RTT = 共 2 RTT。
- **HTTP/3** 首次连接：QUIC 内部集成了 TLS 1.3，握手与密钥协商同时完成 = 共 1 RTT。
- **HTTP/3** 会话恢复：握手信息和应用数据一起发送 = 共 0 RTT。

QUIC 的连接迁移

TCP 用四元组标识连接，手机从 4G 切到 WiFi 时 IP 变了，必须断开重连，经历 TCP 三次握手 + TLS 握手 + 慢启动，用户会感到卡顿。

QUIC 用连接 ID 标识通信双方，不依赖 IP 地址。网络切换后只要保有连接 ID 和 TLS 密钥等上下文信息，就能无缝复用原连接，无需重连。

HTTP/1.1的优化

主要的优化思路：

- 尽量避免发送HTTP请求
- 在需要发送HTTP请求时，考虑如何减少请求次数
- 减少服务器的HTTP响应的数据大小

如何避免发送HTTP请求？

可以利用缓存技术

1.缓存原理

- 客户端将第一次请求及响应数据保存在本地磁盘，以URL为key，响应为value
- 后续相同请求时，先查本地缓存，若找到则直接读取本地数据，避免网络请求

2.缓存过期与验证

- 服务器在响应头中设置过期时间，客户端发现缓存过期后重新发送请求
- 若资源未变更，客户端在请求头中携带 **Etag**（资源摘要），服务器比较后返回 **304Not Modified**（无包体），告知缓存仍然有效

如何减少HTTP请求次数？

三大方法：减少重定向、合并请求、延迟发送

1.减少重定向请求次数

- 重定向原理：资源迁移时，服务器通过**302**响应码和**Location**头告知客户端新URL，客户端需再次请求新URL
- 优化方案：由代理服务器处理重定向，减少客户端与代理服务器之间的消息传递次数

2.合并请求

- 小资源合并：将多个小文件（如图片、JS、CSS）合并为一个文件，减少HTTP请求次数和TCP连接数
 - 例如：使用CSS Image Sprites 合并小图片，或通过Webpack打包JS/CSS
- Base64嵌入：将图片二进制数据用Base64编码后嵌入HTML，随HTML一同发送

3.延迟发送请求

- 按需加载：仅请求当前用户可见的资源，用户滑动页面时再加载后续资源（如图片懒加载）

如何减少HTTP响应数据大小？

两大压缩方式：无损压缩、有损压缩

1.无损压缩

- 原理：压缩后数据可完全恢复，适合文本、程序文件
- 常见算法：Gzip、Brotli，通过去除冗余符号（如空格、换行）和哈夫曼编码实现
- HTTP头字段：
 - 客户端请求头：Accept-Encoding: gzip, deflate, br
 - 服务端响应头：Content-Encoding: gzip

2.有损压缩

- 原理：牺牲部分质量以减少数据量，适合多媒体（图片、音频、视频）
- 图片压缩：使用WebP格式（比PNG更小），或通过质量因子（q）控制压缩比
- 音视频压缩：利用帧间冗余（如H.264 H.265视频编码，AAC、AC3音频编码）

HTTPS RSA握手解析

HTTPS与TLS的作用

HTTP是明文传输，存在三大风险：窃听、篡改、冒充。HTTPS通过在HTTP和TCP之间加入TLS协议来解决这些问题：

- 信息加密：防止被第三方窃取
- 校验机制：防止数据被篡改
- 身份证书：证明服务器身份

TLS握手流程

TLS握手通常经过4个消息（2个RTT），完成后即可进行安全的HTTP通信

核心机制

数字证书验证（CA签名与校验）

为了防止“冒充风险”，服务端会发送数字证书

- CA签发证书过程（服务端）：
 - 1.CA将持有者公钥、用途、颁发者等信息打包
 - 2.对这些信息进行Hash计算，得到Hash值
 - 3.CA使用自己的 私钥加密Hash值，生成数字签名
 - 4.将签名添加到证书文件中
- 客户端校验过程：
 - 1.使用同样的Hash算法计算收到证书的Hash值（H1）
 - 2.使用操作系统/浏览器内置的CA公钥解密证书中的签名，得到Hash值（H2）
 - 3.对比H1和H2，若相同则证书可信

证书信任链

证书通常不是由根CA直接签发，而是通过三层级结构：

- 1.根证书（Root CA）：自签名，预装在操作系统/浏览器中，绝对信任

2.中间证书（Intermediate CA）：由根证书签发，用于隔离风险

3.服务器证书：由中间证书签发，用于具体的域名

为什么要用中间证书？？

为了确保根证书的绝对安全性，将根证书隔离（通常离线保存）。如果中间证书失守，可以吊销重发，不影响根证书

密钥交换（生成会话密钥）

RSA算法的核心在于生成对方共享的对称密钥（会话密钥）；

1.客户端生成一个随机数 Pre-Master Key

2.使用服务端证书中的 公钥 加密 Pre-Master Key，发送给服务端

3.服务端使用 私钥 解密，得到 Pre-Master Key

4.生成 Master Secret：此时客户端和服务端都拥有了三个随机数：

- Client Random
- Server Random
- Pre-Master Key

双方利用这三个随机数，生成最终的会话密钥（Master Secret）

RSA算法的缺陷

不支持前向保密：因为Pre-Master Key 是用公钥加密传输的，如果未来服务器的 私钥泄漏，攻击者可以利用私钥解密历史截获的流量，算出会话密钥，从而破解过去所有的通信记录

改进方案：使用ECDHE密钥协商算法，这种算法支持前向保密

TLS的优化

最基础的TLS1.2版本及以前版本，需要四次握手，也就是通常需要2个RTT（往返时间）

优化（**TLS1.3**）

1-RTT握手：TLS1.3将握手过程压缩，客户端在发送第一个包时就可以直接猜测并发送加密密钥（提前发送密钥参数）。服务器如果同意，可以直接返回加密数据。这样TLS握手本身只需要一个RTT

0-RTT恢复连接：如果客户端之前访问过该服务器并保存了会话票据（Session Ticket），它可以在第一个包中直接发送加密的应用数据，实现“零往返时间”的数据传输。

DDos攻击

一、什么是**DDoS**攻击？

1.1 定义

DDoS（Distributed Denial of Service，分布式拒绝服务）攻击是一种网络攻击方式，攻击者通过控制大量分布在不同地点的设备（通常称为“僵尸网络”）向目标服务器、网络设备或服务发送海量请求，以耗尽其资源，从而使合法用户无法访问目标服务。

1.2 工作原理

DDoS攻击的核心是利用多个来源同时发起攻击，这些来源设备被称为“僵尸”设备，通常是被恶意软件感染的计算机、服务器、物联网设备等。攻击者通过控制这些设备，向目标发送大量的请求或数据包，造成以下几种情况：

- 带宽耗尽：攻击流量超过目标网络的带宽容量。
- 资源耗尽：消耗目标服务器的CPU、内存等资源。
- 应用层攻击：针对特定应用层协议（如HTTP）的攻击，导致应用服务崩溃。

二、常见的**DDoS**攻击类型

2.1 流量攻击

通过发送大量的数据包，耗尽目标网络的带宽。这类攻击包括**UDP**洪泛攻击、**ICMP**洪泛攻击等。

2.2 协议攻击

通过消耗服务器资源或中间设备资源，导致服务不可用。这类攻击包括**SYN**洪泛攻击、**Ping of Death**攻击等。

2.3 应用层攻击

针对特定应用层协议的攻击，通常难以检测，因为这些攻击流量看似合法。这类攻击包括**HTTP**洪泛攻击、**Slowloris**攻击等。

三、防护**DDoS**攻击的方法

3.1 网络层防护

3.1.1 防火墙和入侵检测系统（**IDS/IPS**）

使用防火墙和**IDS/IPS**系统，设置规则过滤可疑流量，阻止已知的攻击模式。

3.1.2 流量清洗

部署流量清洗设备或使用流量清洗服务，识别并清除恶意流量，仅允许合法流量通过。

3.2 应用层防护

3.2.1 WAF（Web应用防火墙）

WAF可以检测和阻止针对Web应用的DDoS攻击，如SQL注入、跨站脚本等。

3.2.2 速率限制

在服务器端设置请求速率限制，防止恶意用户频繁发送请求。

3.3 分布式防护

3.3.1 CDN（内容分发网络）

使用CDN分散流量，将请求分布到多个节点，缓解单点压力。

3.3.2 负载均衡

通过负载均衡器将流量分散到多个服务器，防止单个服务器过载。

3.4 云服务防护

使用云服务提供商的DDoS防护服务，如AWS Shield、Azure DDoS Protection、阿里云DDoS防护等，这些服务提供专业的DDoS防护能力和实时监控。

3.5 监控与响应

3.5.1 实时监控

部署网络监控系统，实时监控流量，及时发现异常流量和潜在攻击。

3.5.2 应急响应

制定应急响应计划，包含攻击检测、流量清洗、系统恢复等步骤，确保在攻击发生时能够快速响应和处理。

浏览器键入URL，发生了什么？

1. URL 解析

浏览器首先会对你输入的 URL 进行解析，补全协议（如 HTTP 或 HTTPS），并提取出主机名（域名）、端口号、路径以及查询参数等信息。

2. DNS 域名解析（寻找服务器 IP）

浏览器需要将人类易读的域名（如 `www.example.com`）转换成计算机能识别的 IP 地址。查找顺序通常是：

检查浏览器自身的缓存。

检查操作系统的缓存以及本地的 `hosts` 文件。

向本地 DNS 服务器（通常由网络运营商 ISP 提供）发起请求。如果本地没有，DNS 服务器会通过递归查询，最终从权威 DNS 服务器获取到目标服务器的真实 IP 地址。

3. 建立 TCP 连接（三次握手）

拿到目标 IP 后，浏览器会与服务器建立可靠的 TCP 连接。经典的“三次握手”过程如下：

第一次握手：客户端发送 SYN 包，表示“我想建立连接”。

第二次握手：服务端回复 SYN+ACK 包，表示“收到，同意连接”。

第三次握手：客户端回复 ACK 包，表示“我也收到了，连接建立成功”。

HTTPS 专属环节：如果使用的是 HTTPS 协议，在 TCP 握手后还会进行 **TLS/SSL** 加密握手，交换数字证书并协商会话密钥，以确保后续数据传输的安全性。

4. 发送 HTTP 请求与服务器响应

连接建立后，浏览器会构造一个 HTTP 请求报文（包含请求方法如 GET/POST、请求头、Cookie 等信息）发送给服务器。服务器接收到请求后，根据业务逻辑进行处理（如查询数据库），然后返回一个 HTTP 响应报文，其中包含状态码（如 200 表示成功）以及请求的资源内容（如 HTML、CSS、JS、图片等）。

5. 浏览器解析与渲染页面（核心环节）

浏览器拿到服务器返回的 HTML 文档后，开始进行页面的构建与绘制：

构建 **DOM** 树：解析 HTML 标签，从上到下生成文档对象模型（DOM）树。

构建 **CSSOM** 树：解析 CSS 样式（包括外部样式表和内部样式），生成 CSS 对象模型（CSSOM）树。

生成渲染树 (**Render Tree**)：将 DOM 树和 CSSOM 树结合，生成只包含可见元素及其样式的渲染树（例如 `display: none` 的元素不会被加入）。

布局 (**Layout**)：计算渲染树中每个节点在屏幕上的确切位置和大小（也叫重排/回流）。

绘制 (**Paint**)：遍历渲染树，调用底层系统绘制各个节点的像素（如颜色、文字、边框等）。

合成 (**Composite**)：如果页面包含多个图层，浏览器会将这些图层进行合成，最终显示在屏幕上。

6. 加载其他资源与断开连接

在解析 HTML 的过程中，如果遇到 `<img>`、`<script>`、`<link>` 等外部资源标签，浏览器会并行发起新的请求去获取这些资源。当所有数据传输完毕后，根据 HTTP 协议版本和头部设置，可能会通过 TCP 四次挥手来断开连接。

connect方法会阻塞，请问有什么方法可以避免其长时间阻塞？

最常用的方法是加定时器；也可以采用非阻塞模式

或者考虑采用异步传输机制，同步传输与异步传输的主要区别在于同步传输中，如果调用 `recvfrom`后会一直阻塞运行，从而导致调用线程暂停运行；异步传输机制则不然，会立即返回。

网络中，如果客户端突然掉线或重启，服务端怎么样才能立即知道？

若客户端掉线或者重新启动，服务端会收到复位信号，每一种tcp/ip的实现不一样，控制机制也不一样

TTL是什么？有什么用处，通常哪些工具会用到它？

TTL是Time To Live，一般是hop count，每经过一个路由就会被减去1，如果它变为0，包就会被丢掉。它的主要目的是防止包在有回路的网络上死转，浪费网络资源。`ping`和 `traceroute`用到它。

网络编程中设计并发服务器，使用多进程 与 多线程，有什么区别？

进程：子进程是父进程的复制品。子进程获得父进程数据空间、堆和栈的复制品

线程：相对与进程而言，线程是一个更加接近于执行体的概念，它可以于同进程的其他线程共享数据，但拥有自己的栈空间，拥有独立的执行序列

两者都可以提高程序的并发度，提高程序运行效率和响应时间

线程和进程在使用上各有优缺点：线程执行开销小，但不利于资源管理和保护；而进程正相反。同时，线程适合于在对称多处理架构机器上运行，而进程则可以跨机器迁移

fork一个子进程后，父进程的全局变量能不能使用？

fork后子进程将会拥有父进程的几乎一切资源，父子进程的都各自有自己的全局变量。不能通用，不同于线程。对于线程，各个线程共享全局变量

TCP三次握手的过程，accept发生在三次握手的哪个阶段

client的connect引起3次握手

server在socket, bind, listen后，阻塞在accept，三次握手完成后，accept返回一个fd，因此accept发生在三次握手之后

tcp流，udp的数据报，之间有什么区别，为什么TCP要叫做数据流？

TCP本身是面向连接的协议，server和client之间要使用TCP，必须先建立连接，数据就在该连接上流动，可以是shuang

TCP本身是面向连接的协议，S和C之间要使用TCP，必须先建立连接，数据就在该连接上流动，可以是双向的，没有边界。所以叫数据流，占系统资源多 UDP不是面向连接的，不存在建立连接，释放连接，每个数据包都是独立的包，有边界，一般不会合并。TCP保证数据正确性，UDP可能丢包，TCP保证数据顺序，UDP不保证

socket在什么情况下可读？

- 1.接收缓冲区有数据，一定可读
- 2.对方正常关闭socket，也是可读
- 3.对于侦听socket，有新连接到达也可读

4.socket有错误发生，且pending

TCP通讯中，select到读事件，但是读到的数据量是0，为什么，如何解决？

select 返回0代表超时。select出错返回-1。select到读事件，但是读到的数据量为0，说明对方已经关闭了socket的读端。本端关闭读即可。当select出错时，会将接口置为可读又可写。这时就要通过判断select的返回值为-1来区分。

IO多路复用优缺点？

IO多路复用优点：

- 1.相比基于进程的模型给程序员更多的程序行为控制。
- 2.IO多路复用只需要一个进程就可以处理多个事件，单个进程内数据共享变得容易，调试也更容易。
- 3.因为在单一的进程上下文当中，所以不会有多进程多线程模型的切换开销。IO多路复用缺点：

- 1.业务逻辑处理困难，编程思维不符合人类正常思维。
- 2.不能充分利用多核处理器。

select机制的缺点

每次调用select，都需要把监听的文件描述符集合fd_set从用户态拷贝到内核态，从算法角度来说就是O(n)的时间开销。每次调用select调用返回之后都需要遍历所有文件描述符，判断哪些文件描述符有读写事件发生，这也是O(n)的时间开销。内核对被监控的文件描述符集合大小做了限制，并且这个是通过宏控制的，大小不可改变(限制为1024)。

epoll的好处

epoll解决了select和poll在文件描述符集合拷贝和遍历上的问题，能够在单个进程当中监听多个文件描述符，并且十分高效。

epoll需要在用户态和内核态拷贝数据么？

在注册监听事件时从用户态将数据传入内核态；当返回时需要将就绪队列的内容拷贝到用户空间。

epoll的实现？

在内核当中是什么样的数据结构进行存储，每个操作的时间复杂度是多少？在内核当中是以红黑树的方式组织监听的事件，查询开销是 $O(\log n)$ 。采用回调的方式检测就绪事件，时间复杂度为 $O(1)$ ；

如果说想要优化这个http的用户的

服务器出现大量close_wait的连接的原因是什么？解决办法？

close_wait状态是在TCP四次挥手的时候收到FIN但是没有发送自己的FIN时出现的，服务器出现大量close_wait状态的原因有两种：

- 服务器内部业务处理占用了过多事件，都没能处理完业务；或者还有数据需要发送；或者服务器的业务逻辑有问题，没有执行close()方法
- 服务器的父进程派生出子进程，子进程继承了socket，收到FIN的时候子进程处理但父进程没有处理该信号，导致socket的引用不为0无法回收

处理办法：

- 停止应用程序
- 修改程序bug

一台机器能够使用的端口号上限是多少？是否可以修改？如果想要用的端口超过这个限制怎么办？

一般是65536。因为TCP的报文头部中源端口号和目的端口号的长度是16位，也就是可以表示 $2^{16}=65536$ 个不同端口号，因此TCP可供识别的端口号最多只有65536个。但是由于0到1023是知名服务端口，所以实际上还要少1024个端口号。

而对于服务器来说，可以开的端口号与65536无关，其实是受限于Linux可以打开的文件数量，并且可以通过MaxUserPort来进行配置

如果突破上限，可以采取以下措施：

- 扩展本地端口范围，比如说在linux系统中修改`net.ipv4.ip_local_port_range`关键字增大可用端口范围
- 绑定多个IP地址：一台机器可以绑定多个公网或内网IP，每个IP独立拥有65536个端口，总连接数 = IP数 * 65536
- 使用连接复用技术，如 HTTP/2、gRPC、WebSocket 长连接，减少频繁建立/断开连接的开销，降低端口消耗。
- 分布式部署，将服务拆分为多个节点，通过负载均衡分发请求，单机压力自然下降